



J.A.R.V.I.S v2.1 — Every Feature, Fully Documented



COMPLETE CAPABILITY REFERENCE — Yeh report live page agentjarvis-v2.netlify.app/features/ ka **exact, section-by-section** documentation hai (aapke 16 screenshots se verify kiya gaya). Saari 15 sections, har feature card, aur har stat included hai — kuch nahi chhoota.

EVERY FEATURE, fully documented.

JARVIS v2.1 ek **next-generation agentic harness** hai — **225K lines of code** across **902 source files** with a **single-agent loop** orchestrating **120+ tools, 116 skills, 6 extensions, 6 watchers, a full MCP client**, and **native Claude Code + OpenAI Codex plugin support**. Here is everything it can do.



300+ Features • 120+ Tools • 116 Skills • 6 Extensions



15 Sections: 01 Four User Interfaces · 02 Agent System · 03 Tool System · 04 Memory & Context · 05 Extension System · 06 Plugin Compatibility · 07 Skills System · 08 Background Watchers · 09 Event & Hook · 10 LLM & Model · 11 MCP Integration · 12 Knowledge Graph · 13 Learning & Evolution · 14 Configuration & Theming · 15 Execution & Safety



Section 01 — Four User Interfaces

TUI, WebUI, RPC, and CLI — all backed by the same agent loop and tool system.



41 Widgets • 15 RPC Handlers • 4 Interfaces • 51 tokens Theme

- **TUI (Terminal UI)** — Textual-based terminal interface with 41 custom widgets: streaming messages, tool approval prompts, model picker, voice app, session picker, rewind app, debug console, context progress bar, clipboard integration.
- **WebUI** — Browser-based interface with FastAPI backend, React/TypeScript + Vite + Tailwind frontend, WebSocket real-time streaming, glass-morphism dark theme, session resume across page reloads.
- **RPC (JSONL)** — JSONL over stdin/stdout for IDE and process embedding. 15 handler modules exposing 40+ commands including `prompt`, `steer`, `follow_up`, `bash`, `compact`, `get_state`, `get_messages`, `set_model`, `batch`. Full streaming event support.
- **CLI & REPL** — Terminal REPL (via `--cli`) with autocomplete history, real-time thought token streaming, interactive tool confirmations, and slash commands (`/help`, `/skills`, `/skill`, `/yolo`, `/status`, `/clear`, `/compact`, `/compress`).
- **ACP Server** — Agent Communication Protocol via `acp-sdk`. HTTP/REST endpoints (`/agents`, `/runs`), three run modes (sync/async/stream), tool trajectory metadata, lazy agent init.



Section 02 — Agent System

6 specialized agent classes with 9 profiles, concurrent tool execution, and heartbeat scheduling.



6 Agent Classes • 9 Profiles • 4 Safety Levels • Async Concurrent Tools

- **JarvisV2 (Primary)** — Main coding, research, and documentation agent. ReAct/tool-calling loop with streaming, concurrent tool execution, context compaction integration, learning system, and heartbeat scheduler.

- **ExploreAgent** — Read-only codebase exploration agent. File reading and searching only — no writes. Architecture analysis and pattern detection. Safe for parallel exploration.
- **PlanAgent** — Task decomposition and planning agent. Read-only analysis with implementation plan generation and dependency identification.
- **ForkSubagent** — Fork conversation context for parallel work with git worktree isolation. Inherits parent context, runs independently.
- **RubberDuckAgent** — Constructive critique and code review agent. Reviews changes for quality, suggests improvements, identifies potential issues.
- **VerificationAgent** — Code verification agent. Runs builds, tests, linters. Produces PASS/FAIL/PARTIAL verdicts with evidence.
- **9 Profiles** — `default`, `plan`, `accept-edits`, `auto-approve`, `explore`, `basher`, `editor`, `researcher`, `code-reviewer`. Each with 4 safety levels (SAFE/NEUTRAL/DESTRUCTIVE/YOLO) and per-tool permission overrides. Cycle with Shift+Tab.
- **Heartbeat Scheduler** — Two-phase nanobot-style periodic awareness. Configurable interval, active hours, skip-when-busy, light context mode. Reads `HEARTBEAT.md` for autonomous tasks.

Section 03 — Tool System (120+)

File ops, code execution, search, web, memory, agents, skills, git, MCP — all with granular permissions.



120+ Tools • 39 Core Tools • 84 Extension Tools • 3 Sandbox Backends

- **File Operations** — `read` (multi-file parallel with offset/limit), `write` (with diff generation), `edit` (text replacement with diff), `ls` (directory listing), `find` (file search by pattern).
- **Code Execution** — `bash` (shell commands with timeout, background support, sandbox wrapping, dangerous pattern detection), `repl` (Python REPL), `run_tests` (test runner with framework detection).

- **Search** — `grep` (ripgrep-backed regex search with Python fallback), `codebase_search` (knowledge graph + indexer with BFS/DFS queries), `tool_search` (semantic embedding-based tool discovery), `session_search` (FTS5 + always-on hybrid vector search over conversation history).
- **Web** — `fetch_webpage` (HTTP content fetching with markdown conversion), `web_search` (web search via Exa/Brave APIs).
- **Memory CRUD** — Full CRUD for persistent memory: create, save, read, list, search, edit, update, delete across 6 memory types (user, feedback, project, project_context, reference, global).
- **Agent Management** — `agents` (delegate to subagents with filtered tool registries), `ask_user_question` (interactive prompts for user input).
- **Skills** — `skills_list` (browse metadata), `skill_view` (full content), `skill_manage` (create/patch/edit/delete), `activate_skill` (load into context).
- **Git & Background** — `enter_worktree` / `exit_worktree` (isolated git worktrees with tmux), `list_background_processes`, `read_background_output`.

Section 04 — Memory & Context

Five-layer memory architecture with hybrid vector+BM25 retrieval, dialog archiving, and LLM-based compaction.



5 Layers • 6 Memory Types • 3 Scopes • 4.7M Embedding Model

- **Layer 1: TokenCompressor** — Proactive heuristic-based token compression. Zero LLM calls, <1ms latency. 5 strategies: truncate, structured_extract, code_aware, deduplicate, list_compress. 50-80% reduction.
- **Layer 2: ToolResultOffloader** — Archives large tool outputs to disk. Recent results get higher threshold. Snippet + file reference replacement in context. Automatic cleanup with retention days.
- **Layer 3: DialogArchiver** — Persists raw messages before compaction as daily JSONL files (`YYYY-MM-DD.jsonl`). Session-filtered retrieval. 30-day default retention.

- **Layer 4: ContextCompaction** — LLM-based summarization at 80% context utilization. 3-layer compression: tool pruning → offloading → LLM summarization. Anti-thrashing, cross-process locks, deterministic fallback.
- **Layer 5: HybridRetriever** — Vector (sentence-transformers VTXAI/Vortex-Embed-4.7M) + BM25 keyword search. Configurable weights (70% vector, 30% BM25). Lazy initialization, index caching.
- **Persistent Memory** — 6 memory types: user, feedback, project, project_context, reference, global. 3 scopes: private, team, global. Markdown with YAML frontmatter. Security scanning for secrets.
- **Session Management** — SQLite-backed storage in OpenAI SDK format. FTS5 full-text search with trigram tokenizer (CJK support). Always-on hybrid search with VTXAI/Vortex-Embed-4.7M vector embeddings for semantic reranking. WAL mode, compression locks, session lineage traversal.
- **Rewind System** — File snapshots before user messages. Message truncation to any point. Session forking from any checkpoint. Full state restoration.

Section 05 — Extension System

6 installed extensions adding 84 tools. Plugin architecture with 14 registration methods.



6 Extensions • 14 API Methods • 84 Extension Tools • 5 Lifecycle Stages

- **Extension API** — 14 registration methods: `tools()`, `agent_tools()`, `agents()`, `agent_loop()`, `skills()`, `command()`, `on()`, `hook()`, `shortcut()`, `tool_icon()`, `tool_result_widget()`, `tool_approval_widget()`, `tool_args_model()`, `load_config()`.
- **Calculator** — Mathematical expression evaluation with restricted eval sandbox. Builtins disabled, no imports. Single calc tool.
- **ML Intern** — Autonomous ML engineering agent for Hugging Face ecosystem. 18+ tools: paper discovery, HF docs, dataset auditing, HF Jobs

(CPU → A100×8), Kaggle competitions, sandbox environments, Trackio monitoring.

- **Mobile (Android)** — 14 tools for Android device automation via uiautomator2. UI hierarchy capture, element interaction, text input, gesture control, hardware buttons, ADB shell. Annotated screenshots.
- **OSINT** — 11 tools for Open Source Intelligence. 6 free (IP geolocation, GitHub, Google dorks, secret scanner, HackerOne, maigret) + 5 API-key (HIBP, VirusTotal, Censys, IP2Location, AbuseIPDB).
- **Trading Analysis** — Multi-agent stock analysis pipeline. 5 phases: Parallel Data Analysis → Adversarial Bull/Bear Debate → Research Manager → Trader → Portfolio Manager. BUY/SELL/HOLD with entry, stop, sizing.
- **Lifecycle** — 3-tier discovery: `.jarvis/extensions/` → `~/.jarvis/extensions/` → pip entry points. discover → load → bind → run → unbind. Tool override detection, conflict logging, MCP server declarations.

Section 06 — Plugin Compatibility Bridge

Run Claude Code and OpenAI Codex plugins natively — 8 auto-discovery sources, 5-gate permission system.



2 Formats • 8 sources Discovery • **5** Permission Gates • **134 exports**
Public API

- **Claude Code Support** — Loads `.claude-plugin/plugin.json` manifests. Translates skills/, agents/, commands/, hooks/, `.mcp.json`, `.lsp.json`, monitors/, themes/, output-styles/ into live ExtensionAPI registrations.
- **OpenAI Codex Support** — Loads `.codex-plugin/plugin.json` with full interface{} metadata block (displayName, capabilities, branding, assets). Translates skills/, `.app.json` (apps/connectors), mcp_servers wrapped form, hooks.
- **Auto-Discovery (8 sources)** — `$JARVIS_PLUGIN_DIRS`, `~/.claude/plugins/cache/`, `~/.codex/plugins/cache/`, `<cwd>/.claude/plugins/`, `<cwd>/.codex/plugins/`,

`~/.claude/skills/`, `~/.claude/plugins/marketplaces/`, `~/.agents/skills/`. Precedence-based dedupe.

- **Env Var Substitution** — `${CLAUDE_PLUGIN_ROOT}`, `${PLUGIN_ROOT}`, `${CLAUDE_PLUGIN_DATA}`, `${PLUGIN_DATA}` resolved at translation time. Full \$VAR support from process env + plugin env. Path traversal prevention.
- **Hook Execution (5-gate)** — Built-in deny list (rm -rf, fork bombs, reverse shells, mkfs) is immutable. Per-plugin user allow/deny globs. Active safety profile respected (explore/plan=block, auto_approve=allow, default=prompt). Subprocess JSON output parsing.
- **Hook Event Mapping** — PreToolUse → BEFORE_TOOL_CALL, PostToolUse → AFTER_TOOL_CALL, UserPromptSubmit → BEFORE_PROMPT_BUILD, SessionStart → AFTER_SESSION_START, Stop → AFTER_AGENT_END, SubagentStart → AFTER_AGENT_START. Notification/PreCompact log-only.
- **Namespacing** — All skills, agents, commands namespaced as `<plugin>:<name>` to prevent collisions with JARVIS built-ins. Plugin subagents have mcpServers/hooks/permissionMode stripped (with warnings) — declared at plugin level instead.
- **MCP & Commands** — MCP servers registered via `api.tools({mcpServers: {...}})` with per-server policy from `~/.codex/config.toml`. Slash commands from `commands/*.md` with `$ARGUMENTS` substitution.

Section 07 — Skills System (116)

116 builtin skills across 26 categories. Dynamic SKILL.md loading with progressive disclosure and security scanning.



116 Builtin Skills • **26** Categories • **4** Trust Levels • **4** Search Paths

- **Skill Architecture** — `SKILL.md` files with YAML frontmatter. Auto-discovered from 4 search paths: builtin (116), `~/.claude/skills`, `~/.agents/skills`,

`~/.jarvis/skills` . Progressive disclosure: list shows metadata, view shows full content.

- **Creative (20)** — architecture-diagram, ascii-art, ascii-video, baoyu-article-illustrator, baoyu-comic, baoyu-infographic, claude-design, comfyui, creative-ideation, design-md, excalidraw, humanizer, manim-video, p5js, pixel-art, popular-web-designs, pretext, sketch, songwriting-and-ai-music, touchdesigner-mcp.
- **Software Dev (12)** — debugging-hermes-tui-commands, jarvis-skill-authoring, hermes-s6-container-supervision, node-inspect-debugger, plan, python-debugpy, requesting-code-review, spike, subagent-driven-development, systematic-debugging, test-driven-development, writing-plans.
- **Productivity (9)** — airtable, google-workspace, linear, maps, nano-pdf, notion, ocr-and-documents, powerpoint, teams-meeting-pipeline.
- **MLOps (8)** — huggingface-hub, lm-evaluation-harness, weights-and-biases, llama-cpp, obliterated, vllm, audiocraft, segment-anything, dsp.
- **GitHub (6)** — codebase-inspection, github-auth, github-code-review, github-issues, github-pr-workflow, github-repo-management.
- **Security & Trust** — 4 trust levels: builtin → trusted → community → agent-created. Skills guard with security scanning. Path traversal prevention. Provenance tracking. Usage telemetry with lifecycle states (active/stale/archived/pinned).
- **Sources** — OpenClaw registry (agentskills.io), GitHub repositories, local filesystem. Manifest-based sync for bundled skills. Install, sync, optimize, bench CLI commands.

Section 08 — Background Watchers

6 active watchers (4 AI-powered, 2 passive) plus 4 example watchers.



6 Active Watchers • 4 AI-Powered • 2 Passive • 147+ Watcher Tools

- **WhatsApp (48 tools)** — Autonomous WhatsApp assistant via Neonize (Whatsmeow). Messaging, reading, presence, media, groups, profile, privacy tools. Per-chat history, contact resolution, message dedup, self-healing reconnection.
- **Telegram (56 tools)** — Autonomous Telegram assistant via Telethon. Messaging (13), chats (11), contacts (8), groups (14), media (3), profile (7). Push updates + polling fallback. Configurable tool sets via env vars.
- **Instagram (43 tools)** — Instagram Graph API integration. Profile/account, media/publishing, comments, DMs, hashtag discovery. Publish media, carousels, reels. Business discovery and insights.
- **Gesture Control** — Control Android phone via hand gestures and eye movements using MediaPipe Tasks API. 6 hand gestures + 4 eye directions + blinks. Sliding window stabilization. Configurable gesture-to-action mappings.
- **Weather (Passive)** — Open-Meteo API integration. Auto-detects location from IP. Windows toast notifications on significant weather changes. Temperature, humidity, wind, weather code tracking.
- **World Monitor (Passive)** — Geopolitical awareness engine. 7 categories, 25+ endpoints: intelligence, conflict, supply chain, climate, markets, economic, infrastructure. MD5 dedup. Telegram notifications.



Section 09 — Event & Hook System

30+ event types across 8 categories. 18 lifecycle hooks with block/modify/inject semantics.



30+ Event Types • 18 Hook Stages • 8 Categories • Polymorphic Dispatch

- **EventBus** — Per-session pub/sub with priority-ordered handlers. 30+ event types: AgentStarted/Ended/Error, TurnStarted/Ended, MessageDelta/Complete, ThinkingDelta, ToolCallStarted/Ended/Error, SessionStarted/Shutdown, ExtensionLoaded/Unloaded, StatusUpdated, ProgressUpdated, SystemWarning.

- **HookRegistry** — 18 lifecycle stages: before/after agent, turn, prompt, tool, session, system prompt, skill. Decorator-based registration. Block/modify/inject semantics via HookResult. Short-circuit on block.
- **Async-aware** — EventBus gathers async handlers, calls sync sequentially. Polymorphic dispatch via MRO walking. Statistics tracking: total emitted, per-type counts, slowest handler identification.
- **Categories** — 8 event categories: Agent, Turn, Message, Tool, Session, Extension, Status, System. Each with typed frozen dataclass events. Priority ordering for handler execution.



Section 10 — LLM & Model System

Multi-provider support with OpenAI and Anthropic SDKs. Model info, capabilities detection, and fuzzy matching.



2+ Providers • 6 Capabilities • 1 hour Cache TTL • Real-time Streaming

- **Multi-Provider** — OpenAI SDK (primary), Anthropic SDK (internal), and any OpenAI-compatible API via `--base_url`. BaseLLMProvider abstract interface with `generate()` and `generate_with_tools()`.
- **SDKAdapter** — Bridges SDK patterns with agent interface. Token usage tracking from streaming responses. Message conversion. Auto-upgrade http → https for gateways. Identity encoding for compatibility.
- **Model Information** — Fetched from models.dev API with 1-hour cache. Capabilities: reasoning, tool_call, temperature, structured_output, vision, audio. Context/output limits, pricing, modalities. Fuzzy matching.
- **Streaming** — Real-time token-by-token streaming across all interfaces. Thinking/reasoning delta support. Tool call streaming with partial arguments. Progress updates for long operations.



Section 11 — MCP Integration

Full Model Context Protocol client with lazy connections, proxy tool pattern, and OAuth authentication.



3 Transports • 3 Auth Methods • 3 Lifecycle Modes • 4 servers
Configured

- **Lazy Architecture** — Single `mcp` proxy tool for all MCP server tools. Servers connect only when their tools are first called. Direct tool promotion for high-frequency tools.
- **Lifecycle Modes** — lazy (connect on first call, disconnect after idle), eager (connect at startup), keep-alive (connect at startup, auto-reconnect on failure). Configurable idle timeout.
- **Transports** — stdio (subprocess), HTTP (POST to endpoint), SSE (Server-Sent Events streaming). Full bidirectional communication support.
- **Authentication** — OAuth2 (full flow with token refresh), Bearer (static token in header), API Key (header or query parameter). Metadata cache for offline discovery.
- **Features** — Resource support (read MCP resources), prompt templates (MCP-defined), sampling protocol, tool adapter (wraps MCP as BaseTool), MCPRegistry server management.



Section 12 — Knowledge Graph

Full knowledge graph system for codebase understanding — build, analyze, query, visualize, and serve.



20+ Modules • BFS/DFS Query Types • HTML Visualizations • MCP
Integrations

- **Graph Construction** — Build knowledge graphs from codebase AST. Entity extraction, pattern detection, community clustering, deduplication, validation. BFS/DFS queries, shortest path, community detection.

- **Visualization** — Call flow HTML renderings of function call chains. Tree HTML renderings of directory/class hierarchies. Live watch mode for graph updates.
- **Analysis** — PR analysis, wiki generation, report generation. Benchmarking and diagnostics. Google Workspace integration. Audio transcription support.
- **MCP Server** — Serve knowledge graph via MCP protocol for external queries. MCP data ingestion for combining multiple data sources.



Section 13 — Learning & Evolution

M1 → M2 → M3 distillation pipeline, pattern detection, skill crystallization, and GEPA prompt evolution.



M1 → M2 → M3 Pipeline • GEPA Evolution • 50-80% Compression • <1ms Latency

- **Self-Learning** — M1 → M2 → M3 distillation: trace logging → pattern detection → skill crystallization. Auto-creates skills after N repeated patterns. Self-evaluation every N cycles. ML-based intent classification.
- **GEPA Evolution** — DSPy + Genetic-Pareto Prompt Evolution. Optimizes system prompts and skills based on interaction telemetry. Fitness evaluation with Pareto-optimal selection.
- **Safety Guardrails** — Size limits on evolved prompts. Growth caps per evolution cycle. YAML structure integrity checks. pytest gate — must pass tests before applying changes.
- **TokenJuice (Experimental)** — Proactive heuristic token compression. Zero LLM calls, <1ms latency. 50-80% reduction. 5 strategies, 3 compression levels. Configurable min token threshold.



Section 14 — Configuration & Theming

Layered configuration with Pydantic models, 51-token theme system, and namespaced keybindings.



51 Color Tokens • 30+ Key Bindings • 5 Namespaces • 5 Config Layers

- **Layered Config** — Precedence: `defaults` < `~/.jarvis/settings.json` < `.jarvis/settings.json` < env vars < runtime. Deep merge with unknown key preservation. File-lock for concurrent access. Pydantic models.
- **Theme System** — 51 color tokens: core UI, messages, markdown, syntax highlighting, thinking levels, UI elements. Dark and light built-in themes. Custom JSON themes. Truecolor/256-color detection.
- **Keybinding System** — 5 namespaces: editor, input, agent, session, view. 30+ default bindings. JSON config with hot-reload. Legacy ID migration. Customizable per-user.
- **Settings Sections** — tools (permissions, allowlists, denylists), `async_settings` (concurrency, timeouts), heartbeat, learning, sandbox, memory (compaction, retrieval), experimental, watcher, extension.



Section 15 — Execution & Safety

Sandboxed execution with 3 backends, conversation rewind, dangerous pattern detection, and trusted folders.



3 Sandboxes • 3 Permission Levels • 5 Scopes • Sensitive Patterns

- **Sandbox Backends** — Bubblewrap (bwrap) for Linux namespace sandboxing, OpenSandbox for HTTP API sandbox server, HF Space for remote cloud execution. Supported tools: `bash`, `run_tests`.
- **Permission System** — 3 levels per tool: ALWAYS, NEVER, ASK. Granular scopes: `command_pattern`, `outside_directory`, `file_pattern`, `url_pattern`, `sensitive_file`. Trusted folders (TOML-based). Session rules.
- **Rewind System** — File snapshots before each user message. Message truncation to any point. Session forking from any checkpoint. Full state restoration to snapshot.

- **Dangerous Pattern Detection** — Bash tool detects `rm -rf`, `chmod 777`, piping to `sh/bash`, network ops to unknown hosts, credential file access, system file modifications. Automatic warnings.
-



Saari 15 sections cover ho gayi — Interfaces, Agents, Tools, Memory, Extensions, Plugin Bridge, Skills, Watchers, Events/Hooks, LLM, MCP, Knowledge Graph, Learning, Config/Theming, aur Execution/Safety. Yeh live [/features/](#) page ka complete mirror hai.